DIVA

VAPT Report

Android Application Security Assessment VAPT Report

Scope	Android Mobile Application
APK Name	DIVA (Damn Insecure and Vulnerable App)
Package Name	jakhar.aseem.diva
Date	17/10/2026
Prepared By	Mohamed Saad Master
Reviewer	Jeel Vekariya

Table of Contents

Disclaimer 1

Overview 2

- Pre-Engagement Activities 2
- Engagement Phase: Discovery & Attack / Execution 2
- Tools Used During the Assessment 3
- Document Authorization 4

OWASP Mobile Top 10 Testing Standards References 4

Summary of Vulnerabilities Found 5

Detailed Findings 6

Finding 1 Insecure Logging 6

Finding 2 Hardcoding Issues - Part 1 7

Finding 3 Insecure Data Storage - Part 1 8

Finding 4 Insecure Data Storage - Part 2 9

Finding 5 Insecure Data Storage - Part 3 10

Finding 6 Insecure Data Storage - Part 4 11

Finding 7 Input Validation Issues - Part 1 (SQL Injection) 12

Finding 8 Input Validation Issues - Part 2 (WebView URI Loading) 13

Finding 9 Access Control Issues - Part 1 14

Finding 10 Access Control Issues - Part 2 15

Finding 11 Access Control Issues - Part 3 16

Finding 12 Hardcoding Issues - Part 2 17

Finding 13 Input Validation Issues - Part 3 (Native Buffer Overflow) 18

Conclusion 19

Disclaimer

This report documents an independent, self-directed Android application security assessment conducted by Mohamed Saad Master against DIVA (Damn Insecure and Vulnerable App), a deliberately vulnerable open-source Android application used for hands-on mobile application security training.

This document was prepared for personal learning and portfolio purposes, to demonstrate practical skills in Android penetration testing, including static analysis, dynamic analysis, and reverse engineering.

The findings documented in this report are based on testing performed against the publicly available DIVA Beta APK, package jakhar.aseem.diva, in an isolated Android emulator environment (NoxPlayer). They do not reflect any production system, third-party service, or real user data.

All vulnerabilities documented here are intentionally present in DIVA by design, for educational purposes, and are part of the publicly available DIVA training project. No unauthorized system was accessed during this assessment.

Any changes made to the application after the completion of this testing may impact the accuracy or relevance of these findings.

Mohamed Saad Master

Independent Android Security Assessment — Training / Portfolio Project

Overview

DIVA (Damn Insecure and Vulnerable App) is an Android application that is intentionally built with common mobile security flaws, for the purpose of teaching and practicing Android application penetration testing.

This assessment was carried out independently as a hands-on exercise in mobile application security, with the objective of identifying, exploiting, and documenting each of DIVA's built-in vulnerabilities, following an approach aligned with recognized industry standards and best practices, including:

- OWASP Mobile Top 10
- Secure Coding Practices
- Android Security Guidelines

The primary objectives of the assessment were to:

- Identify security vulnerabilities within the mobile application
- Review how sensitive data is stored and handled
- Evaluate authentication and authorization mechanisms
- Assess the application's resistance to reverse engineering
- Analyze permissions, configurations, and overall security controls
- Practice translating hands-on exploitation into clear, structured reporting

This exercise reflects the kind of resilience assessment a real-world mobile application would undergo against unauthorized access, data exposure, reverse engineering attempts, and misuse of privileges.

Pre-Engagement Activities

Before testing began, the following preparation steps were completed:

- The DIVA Beta APK was downloaded and installed on a NoxPlayer Android emulator.
- JADX-GUI was set up for static source code review and decompilation.
- ADB (Android Debug Bridge) was configured for shell access, log capture, and intent-based interaction with the app.
- Each of DIVA's 13 in-app challenges was reviewed to define the scope and objective of testing.
- A working folder was set up to capture screenshots and terminal output as evidence for each finding.

Engagement Phase: Discovery & Attack / Execution

The security assessment was performed in multiple phases to ensure comprehensive coverage.

1. Static Analysis (SAST)

- Decompiling the APK using JADX-GUI
- Reviewing Java source code for each vulnerable Activity
- Identifying hardcoded credentials and secrets
- Reviewing AndroidManifest.xml for exported components
- Reviewing strings.xml for resource values referenced by vulnerable logic

2. Dynamic Analysis (DAST)

- Monitoring runtime behavior via adb logcat
- Interacting with each of DIVA's vulnerable activities through the emulator UI
- Testing SQL Injection payloads against the search functionality
- Testing arbitrary URL/URI loading in the WebView component

3. Reverse Engineering & Exploitation

- Extracting SharedPreferences, SQLite databases, and temp/external files via adb shell
- Invoking exported activities and content providers directly using adb shell am start and adb shell content query
- Extracting a secret from a compiled native library using the public DIVA GitHub repository
- Triggering a native buffer overflow by supplying oversized input to a vulnerable native function

Tools Used During the Assessment

- JADX-GUI (APK decompilation and source code review)
- ADB — Android Debug Bridge (shell, logcat, intents, content queries)
- NoxPlayer Android Emulator (runtime / dynamic testing environment)
- SQLite3 CLI (local database inspection)
- GitHub (public DIVA source repository)
- Windows Command Prompt

Document Authorization

Title	Version	Author	Reviewer	Date
DIVA Android Application VAPT Report	1.0	Mohamed Saad Master	Self-Reviewed	17/10/2026

OWASP Mobile Top 10 Testing Standards References

Category	Test Name
M1	Improper Credential Usage
M2	Inadequate Supply Chain Security
M3	Insecure Authentication / Authorization
M4	Insufficient Input/Output Validation
M5	Insecure Communication
M6	Inadequate Privacy Controls
M7	Insufficient Binary Protections
M8	Security Misconfiguration
M9	Insecure Data Storage
M10	Insufficient Cryptography

Severity Legend:

Critical	High	Medium	Low	Informational
9.0 - 10.0	**7.0 - 8.9**	**4.0 - 6.9**	**0.1 - 3.9**	**N/A**

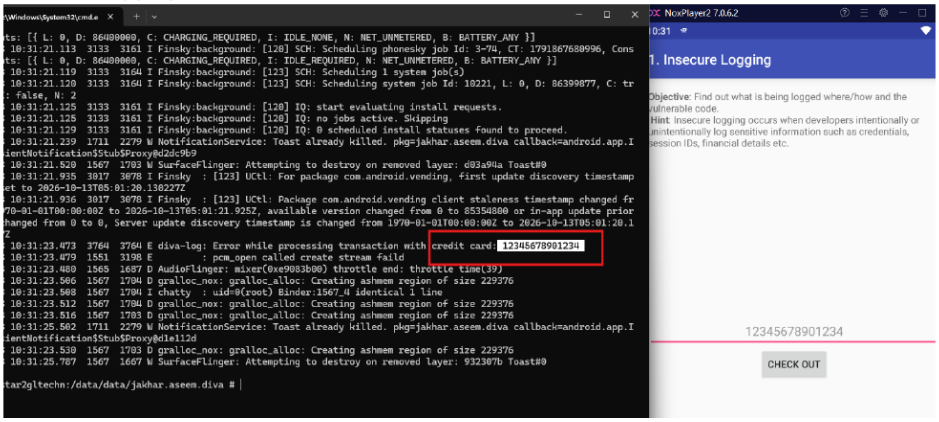
Summary of Vulnerabilities Found

No.	Vulnerability Name	CVSS Score	Severity	OWASP Category
1	Insecure Logging	5.5 (Medium)	Medium	M9 Insecure Data Storage
2	Hardcoding Issues - Part 1	7.5 (High)	High	M1 Improper Credential Usage
3	Insecure Data Storage - Part 1	7.1 (High)	High	M9 Insecure Data Storage
4	Insecure Data Storage - Part 2	7.1 (High)	High	M9 Insecure Data Storage
5	Insecure Data Storage - Part 3	6.5 (Medium)	Medium	M9 Insecure Data Storage
6	Insecure Data Storage - Part 4	7.1 (High)	High	M9 Insecure Data Storage
7	Input Validation Issues - Part 1 (SQL Injection)	8.2 (High)	High	M4 Insufficient Input/Output Validation
8	Input Validation Issues - Part 2 (WebView URI Loading)	7.4 (High)	High	M4 Insufficient Input/Output Validation
9	Access Control Issues - Part 1	7.1 (High)	High	M3 Insecure Authentication/Authorization
10	Access Control Issues - Part 2	7.1 (High)	High	M3 Insecure Authentication/Authorization
11	Access Control Issues - Part 3	7.8 (High)	High	M3 Insecure Authentication/Authorization
12	Hardcoding Issues - Part 2	7.5 (High)	High	M7 Insufficient Binary Protections
13	Input Validation Issues - Part 3 (Native Buffer Overflow)	7.3 (High)	High	M4 Insufficient Input/Output Validation

Detailed Findings

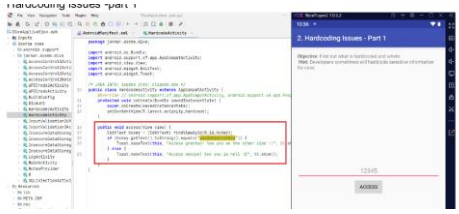
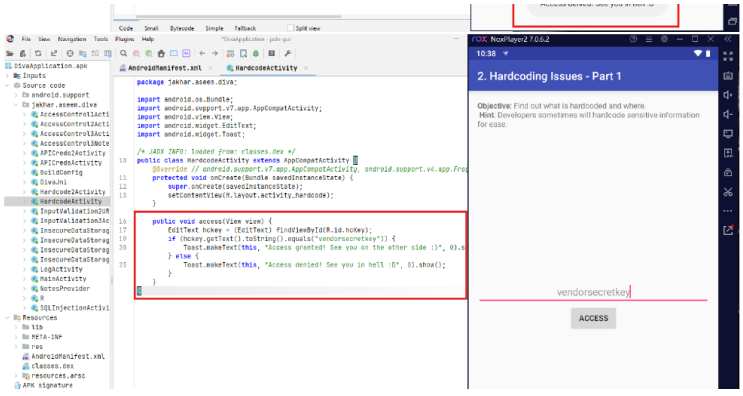
Finding 1 Insecure Logging

Vulnerable Component	Application Logs (Logcat)
OWASP Reference	M9 Insecure Data Storage
CVSS Score	**5.5 (Medium)**
Severity	**Medium**

Summary	During the checkout flow, the application writes the user's credit card number directly into the system log via a plain error-log statement ("diva-log: Error while processing transaction with credit card: <value>"). Android log output is not encrypted or restricted to the app's own process on a debuggable/rooted device, so this data is fully captured by simply running adb logcat while the transaction is performed.
Impact	• Credit card numbers and other transaction data are exposed to anyone with adb/logcat access to the device. • Log data can be collected by other apps with log-read access on older or rooted devices, or by anyone with physical/USB debugging access. • Persisted log buffers extend the window during which sensitive data remains recoverable.
Proof of Concept (POC)	1. Insecure logging  *Figure 1.1 — adb logcat capturing the credit card number in plain text during checkout.*
Mitigation	• Never log PII, credentials, or payment data, even at debug level. • Strip or mask sensitive fields before any log call. • Disable verbose/debug logging in release builds and enable log stripping (e.g., ProGuard/R8).

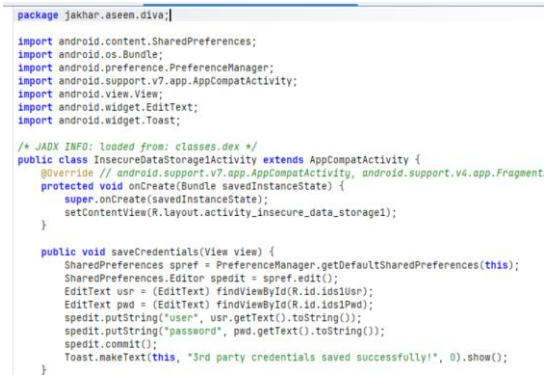
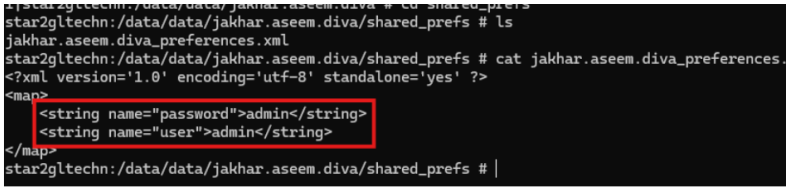
Finding 2 Hardcoding Issues – Part 1

Vulnerable Component	HardcodeActivity (Java source)
OWASP Reference	M1 Improper Credential Usage
CVSS Score	**7.5 (High)**
Severity	**High**

Summary	The “vendor key” that gates access on this screen is hardcoded as a plaintext string literal ("vendorsecretkey") directly inside HardcodeActivity.java. Decompiling the APK with JADX-GUI immediately reveals the literal being compared against user input with a simple .equals() check, so the protection is defeated just by reading the source.
Impact	• Anyone who downloads the APK can recover the secret without ever running the app. • Because the check happens entirely client-side, the “access control” it provides is not a real security boundary. • The same pattern, if reused for a real credential (API key, license key, admin bypass), would fully compromise that control.
Proof of Concept (POC)	 *Figure 2.1 — JADX-GUI view of HardcodeActivity.java with the hardcoded “vendorsecretkey” literal highlighted.*  *Figure 2.2 — the app returning “Access granted!” once that hardcoded value is entered.*
Mitigation	• Never embed secrets, keys, or passwords in client-side source code. • Move authorization decisions to a trusted backend. • If a local check is unavoidable, pair obfuscation with a server-verified token — obfuscation alone is not a security control.

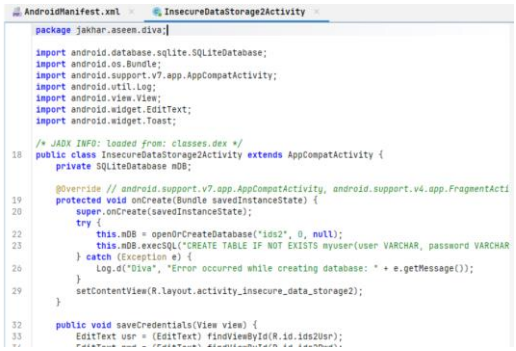
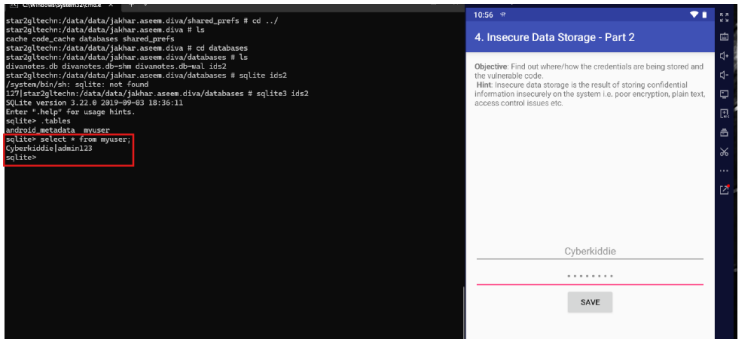
Finding 3 Insecure Data Storage – Part 1

Vulnerable Component	SharedPreferences (jakhar.aseem.diva_preferences.xml)
OWASP Reference	M9 Insecure Data Storage
CVSS Score	**7.1 (High)**
Severity	**High**

Summary	Third-party username and password values entered on this screen are saved via PreferenceManager.getDefaultSharedPreferences() with no encryption. The resulting XML file is created under the app's /shared_prefs directory, and both the "user" and "password" values are stored as plain text — readable in full with a simple cat command.
Impact	• Any process with access to the app's private storage (root access, backup extraction, or a malicious app exploiting another vulnerability) can recover the credentials outright. • No decryption or additional effort is required once file access is obtained.
Proof of Concept (POC)	 *Figure 3.1 — InsecureDataStorage1Activity.java writing the username and password to SharedPreferences with no encryption.*  *Figure 3.2 — adb shell cat of jakhar.aseem.diva_preferences.xml showing the plaintext user and password values.*
Mitigation	• Use EncryptedSharedPreferences (Jetpack Security) or the Android Keystore for anything sensitive. • Avoid persisting third-party credentials on-device where possible; prefer short-lived tokens instead.

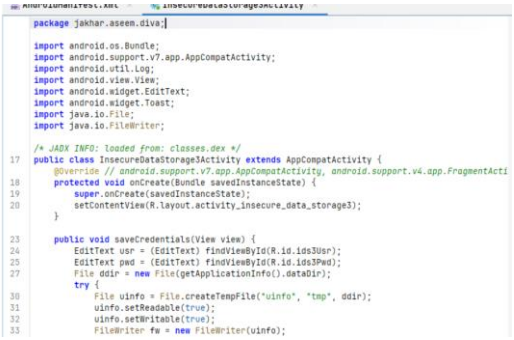
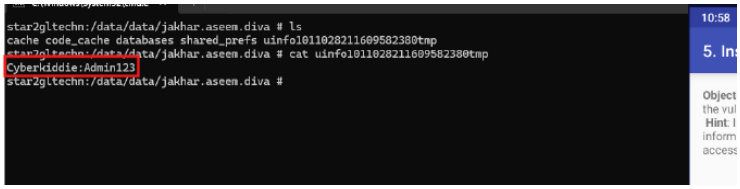
Finding 4 Insecure Data Storage – Part 2

Vulnerable Component	SQLite database (ids2)
OWASP Reference	M9 Insecure Data Storage
CVSS Score	**7.1 (High)**
Severity	**High**

Summary	This screen persists the same type of credentials into a local SQLite database (openOrCreateDatabase("ids2", ...)) with no encryption applied to the database file or its contents. Querying the table directly with the sqlite3 CLI against the extracted database file returns the username and password in plain text.
Impact	• Any actor with file-system access to the app's data directory can dump the full credentials table without interacting with the app itself. • Equivalent exposure to Finding 3, but via a structured database rather than a flat file.
Proof of Concept (POC)	 *Figure 4.1 — InsecureDataStorage2Activity.java inserting the credentials into the local SQLite database in plain text.*  *Figure 4.2 — sqlite3 "select * from myuser;" returning the stored username and password unencrypted.*
Mitigation	• Encrypt the database file (e.g., SQLCipher) or encrypt sensitive columns before insert. • Apply least-privilege access and avoid storing raw credentials client-side.

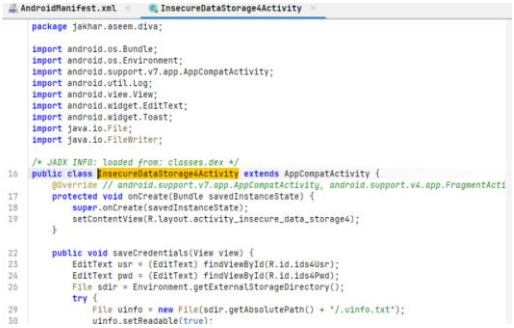
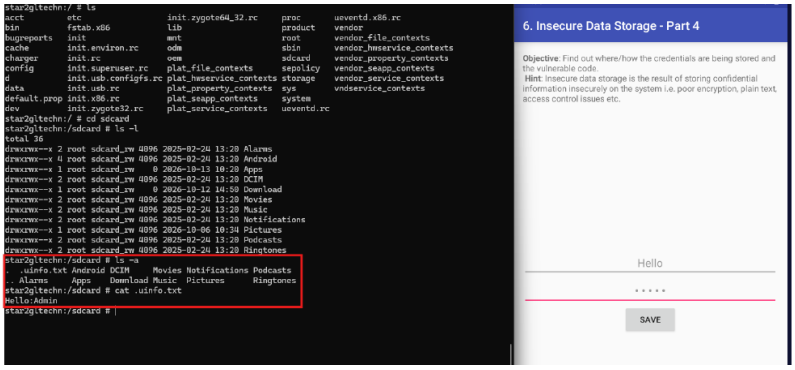
Finding 5 Insecure Data Storage – Part 3

Vulnerable Component	Internal storage temp file (File.createTempFile)
OWASP Reference	M9 Insecure Data Storage
CVSS Score	6.5 (Medium)
Severity	Medium

Summary	Credentials entered here are written with File.createTempFile() into the app's private internal storage, and the code explicitly calls setReadable(true) / setWritable(true) on the resulting file, loosening its permissions beyond the app's own sandbox. Although the generated filename is randomized, the file can still be located and read once shell/file access to the app's data directory is available.
Impact	• Discloses stored credentials to any process able to browse the app's private directory. • The broadened file permissions compound the exposure beyond what the Android sandbox would otherwise allow.
Proof of Concept (POC)	 *Figure 5.1 — InsecureDataStorage3Activity.java writing credentials to a temp file and relaxing its permissions with setReadable/setWritable(true).*  *Figure 5.2 — locating and reading the randomly-named temp file via ls and cat.*
Mitigation	• Never relax default file permissions (setReadable/setWritable(true)) on files containing sensitive data. • Encrypt file contents, or avoid writing credentials to disk entirely.

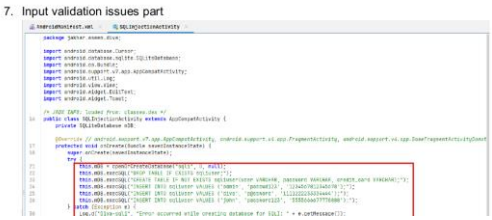
Finding 6 Insecure Data Storage – Part 4

Vulnerable Component	External storage (.uinfo.txt on /sdcard)
OWASP Reference	M9 Insecure Data Storage
CVSS Score	**7.1 (High)**
Severity	**High**

Summary	This screen writes the entered credentials to a hidden dotfile (.uinfo.txt) on external storage via Environment.getExternalStorageDirectory(), once the Storage permission is granted. External storage is world-readable to any app holding the storage permission (and is reachable over USB/file managers), so the file is recovered with ls -a followed by cat.
Impact	• Any other application on the device with storage access — not just DIVA itself — can read the credentials. • Data placed on external storage persists outside the app's own sandbox and can survive app uninstallation, extending the exposure window.
Proof of Concept (POC)	 *Figure 6.1 — InsecureDataStorage4Activity.java writing credentials to external storage via getExternalStorageDirectory().*  *Figure 6.2 — locating the hidden .uinfo.txt file on /sdcard with ls -a and reading it with cat.*
Mitigation	• Avoid external storage for any sensitive data; use app-private internal storage or scoped storage APIs. • Encrypt data if external storage use is unavoidable.

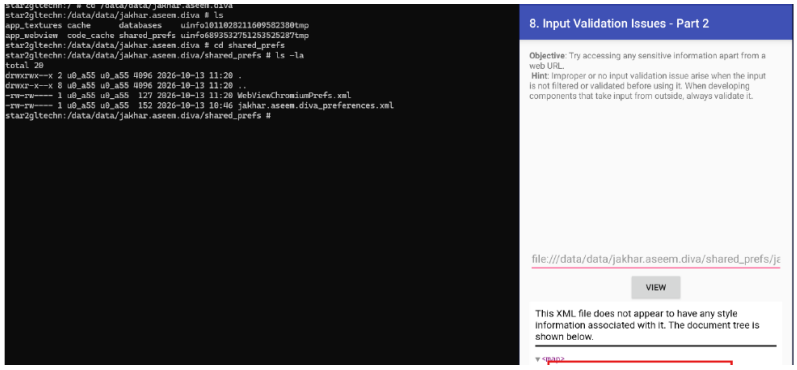
Finding 7 Input Validation Issues – Part 1 (SQL Injection)

Vulnerable Component	SQLInjectionActivity – search field
OWASP Reference	M4 Insufficient Input/Output Validation
CVSS Score	**8.2 (High)**
Severity	**High**

Summary	The user search field builds a raw SQL query by directly concatenating user input ("SELECT * FROM sqliuser WHERE user = '" + input + "'") with no parameterization or sanitization. Submitting the payload ' OR 1 -- returns every row in the table in a single request.
Impact	• A single unauthenticated query dumps the entire user table, including usernames, passwords, and credit card numbers for every seeded account. • Represents a complete breach of data confidentiality for that table from a single input field.
Proof of Concept (POC)	 *Figure 7.1 — SQLInjectionActivity.java building the query by string concatenation, with the seeded user table visible in the source.*  *Figure 7.2 — the app returning all three users' credentials and card numbers from a single ' OR 1 -- search.*
Mitigation	• Use parameterized queries / prepared statements (bind arguments) everywhere user input reaches SQL. • Apply allow-list input validation as a defense-in-depth layer on top of parameterization.

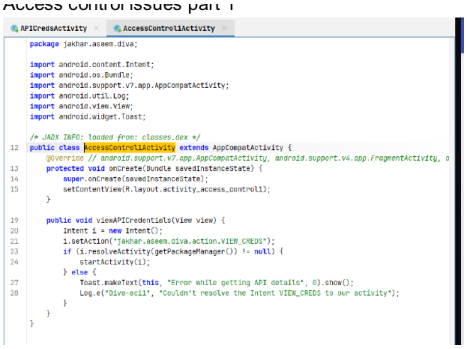
Finding 8 Input Validation Issues – Part 2 (Arbitrary WebView URI Loading)

Vulnerable Component	InputValidation2URISchemeActivity (WebView)
OWASP Reference	M4 Insufficient Input/Output Validation
CVSS Score	**7.4 (High)**
Severity	**High**

Summary	The screen loads whatever URL or URI the user types directly into a WebView via wview.loadUrl(uriText.getText().toString()), with JavaScript enabled and no restriction on scheme or origin. Supplying a local file:// path to the app's own SharedPreferences XML causes the WebView to render the file's raw contents, including the plaintext credentials from Finding 3.
Impact	• Beyond standard web risks (phishing, drive-by content), the unrestricted file:// scheme turns this field into a local file-disclosure primitive. • Exposes any file the app's WebView process is able to read on the device.
Proof of Concept (POC)	 *Figure 8.1 — InputValidation2URISchemeActivity.java loading unsanitized user input directly into the WebView via loadUrl().*  *Figure 8.2 — the WebView rendering the app's own SharedPreferences XML via a local file:// URI, revealing stored credentials.*
Mitigation	• Validate and allow-list the schemes/hosts a WebView may load; reject file:// and content:// from user input. • Disable setAllowFileAccess / setAllowFileAccessFromFileURLs unless explicitly required. • Avoid passing unsanitized user input directly to loadUrl().

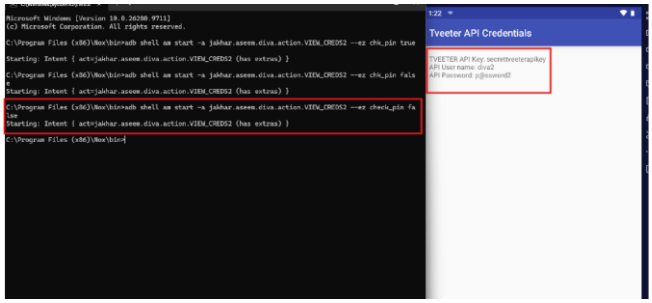
Finding 9 Access Control Issues – Part 1

Vulnerable Component	AccessControl1Activity (exported Activity)
OWASP Reference	M3 Insecure Authentication/Authorization
CVSS Score	**7.1 (High)**
Severity	**High**

Summary	The screen intended to require tapping “View API Credentials” inside the app is backed by an exported Activity reachable directly via an implicit intent. Running adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS launches the credentials screen directly from the command line, with no interaction with the app's own UI and no authorization check in between.
Impact	• Any app or process on the device — not just the legitimate DIVA UI — can trigger this component and read the disclosed vendor API key, username, and password. • No developer-intended flow or gate is actually enforced.
Proof of Concept (POC)	 *Figure 9.1 — AccessControl1Activity.java / APICredsActivity.java showing the exported component's intent action string.*  *Figure 9.2 — adb shell am start directly invoking the exported activity and returning the Vendor API credentials.*
Mitigation	• Set android:exported="false" on activities with no legitimate reason to be externally invokable. • If external invocation is required, enforce a signature-level permission or equivalent caller check inside the component.

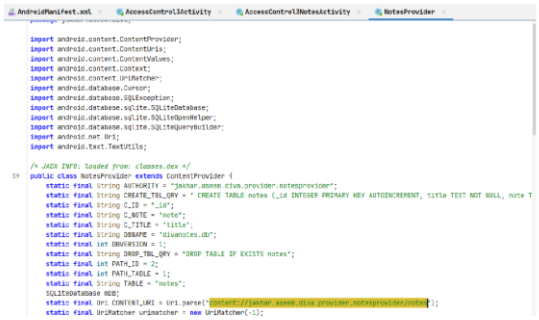
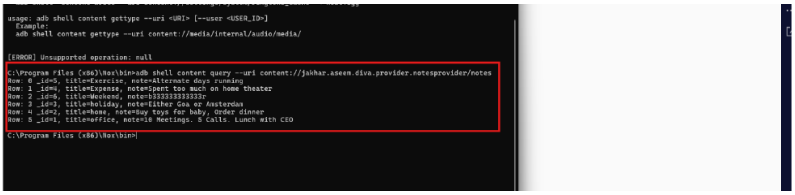
Finding 10 Access Control Issues – Part 2

Vulnerable Component	AccessControl2Activity / APICreds2Activity (exported Activity, boolean extra)
OWASP Reference	M3 Insecure Authentication/Authorization
CVSS Score	7.1 (High)
Severity	High

Summary	This flow is meant to require a PIN obtained by registering with a third party ("Tveeter"). The target activity is again exported and reads a boolean intent extra to decide whether a valid PIN was supplied. Reviewing strings.xml showed the extra's real key is the string resource check_pin rather than the chk_pin value used in a first attempt; once the correct key was supplied with a false value (adb shell am start -a jakhar.aseem.diva.action.VIEW_CREDS2 --ez check_pin false), the credentials screen returned the Tveeter API key and password without ever registering or supplying a PIN.
Impact	• The business-logic gate can be bypassed entirely from outside the app, defeating the intended registration/PIN workflow. • Discloses the third-party API credentials with no legitimate authorization step completed.
Proof of Concept (POC)	 *Figure 10.1 — strings.xml revealing that the chk_pin extra's real key is the resource name check_pin.*  *Figure 10.2 — the sequence of am start attempts, with check_pin false returning the Tveeter API key, username, and password.*
Mitigation	• Never rely on a client-supplied intent extra to gate access to sensitive data; perform the check server-side or against securely stored local state. • Export only the components that truly need to be externally reachable, and validate all intent extras defensively.

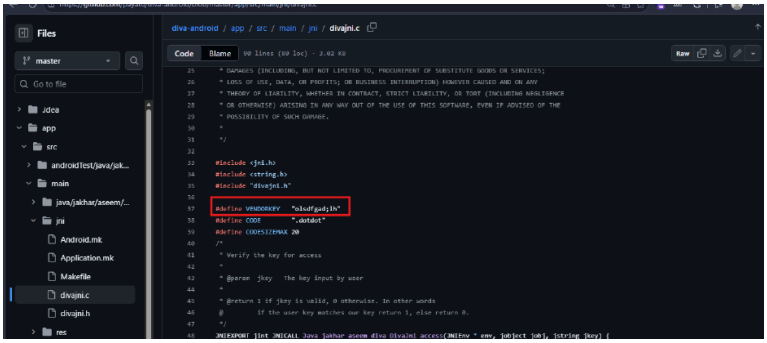
Finding 11 Access Control Issues – Part 3

Vulnerable Component	NotesProvider (exported ContentProvider)
OWASP Reference	M3 Insecure Authentication/Authorization
CVSS Score	**7.8 (High)**
Severity	**High**

Summary	This feature is a PIN-protected private notes app backed by a ContentProvider (content://jakhar.aseem.diva.provider.notesprovider/notes) that is exported with no readPermission defined. The in-app PIN check only gates the activity's own UI — the underlying provider enforces no access control of its own, so its data can be queried directly with adb shell content query, returning every private note in full without ever entering a PIN.
Impact	• Complete disclosure of the user's private notes to any process capable of invoking content query. • The PIN protection is cosmetic: it protects the screen, not the underlying data.
Proof of Concept (POC)	 *Figure 11.1 — NotesProvider.java showing the exported ContentProvider's authority and content URI.*  *Figure 11.2 — adb shell content query returning all private note rows with no PIN supplied.*
Mitigation	• Define and enforce android:readPermission / android:writePermission on any exported ContentProvider. • Perform authorization checks inside the provider's query()/insert() methods, not only in the UI layer that calls them. • Set exported="false" unless another application genuinely needs to use the provider.

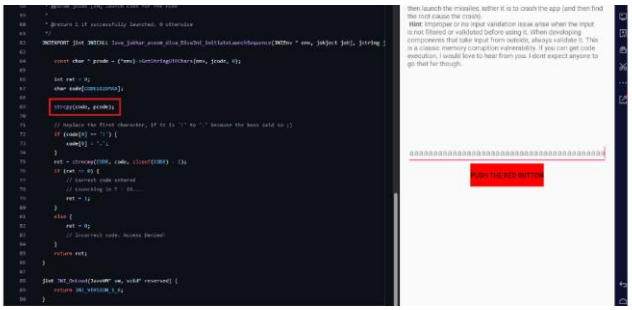
Finding 12 Hardcoding Issues – Part 2

Vulnerable Component	libdivajni.so (native JNI library)
OWASP Reference	M7 Insufficient Binary Protections
CVSS Score	**7.5 (High)**
Severity	**High**

Summary	This vendor key check is implemented in native C code (divajni.c) compiled into libdivajni.so, rather than in Java, specifically to resist decompilation with JADX, which cannot turn native ARM/x86 code back into readable source. However, because DIVA's source repository is public on GitHub, the key (#define VENDORKEY "olsdfgad;lh") was recovered simply by locating app/src/main/jni/divajni.c in the public repository — no binary reverse engineering was required. The same string is also directly recoverable with the strings utility against the compiled .so file, without needing the original source.
Impact	• Moving a secret from Java into native code raises the effort needed to extract it but does not eliminate the risk. • A leaked or public source repository, or straightforward binary reverse engineering (e.g., with Ghidra/IDA), both fully defeat this control.
Proof of Concept (POC)	 *Figure 12.1 — Hardcode2Activity.java / DivaJni.java showing the native method declarations backing the vendor-key check.*  *Figure 12.2 — the public DIVA GitHub repository showing divajni.c with the hardcoded VENDORKEY macro.*
Mitigation	• Do not rely on “security by native code” — treat compiled secrets as recoverable. • Keep real secrets server-side; use native code only to raise the cost of casual extraction, never as the sole protection. • Ensure internal or private forks/build repositories containing real secrets are never made public.

Finding 13 Input Validation Issues – Part 3 (Native Buffer Overflow)

Vulnerable Component	DivaJni.initiateLaunchSequence (native JNI code)
OWASP Reference	M4 Insufficient Input/Output Validation
CVSS Score	**7.3 (High)**
Severity	**High**

Summary	The “launch code” entered on this screen is copied into a fixed-size stack buffer, char code[CODESIZEMAX] (20 bytes), using strcpy(code, pcode) with no bounds checking on the length of the attacker-controlled input. Supplying a string of roughly 40+ characters overflows the buffer and crashes the application (“Diva has stopped”), confirming a classic stack-based buffer overflow in native code.
Impact	• At minimum, this is a reliable denial-of-service (app crash) triggerable by any local input. • Because the overflow occurs in native code with no bounds checking, it is also a potential memory-corruption / code-execution primitive, depending on stack layout and mitigations (ASLR, stack canaries) present in the shipped build.
Proof of Concept (POC)	 *Figure 13.1 — InputValidation3Activity.java / DivaJni.java showing the Java entry point into the native launch-sequence check.*  *Figure 13.2 — the vulnerable strcpy(code, pcode) call in divajni.c, and the oversized input that crashes the app.*
Mitigation	• Replace strcpy with a bounds-checked copy (strncpy/strlcpy) that respects the destination buffer size, and always null-terminate. • Validate and truncate input length in Java before it ever reaches native code. • Build native libraries with stack protector, ASLR, and PIE enabled, and fuzz-test any native code path that parses external input.

Conclusion

This assessment of DIVA (Damn Insecure and Vulnerable App) identified 13 distinct vulnerabilities spanning insecure logging, insecure data storage, insufficient input validation, broken access control, and hardcoded credentials — each representative of a security flaw commonly seen in real-world Android applications.

Working through the application end-to-end, from static source review through dynamic exploitation and native reverse engineering, reinforced the practical importance of secure coding fundamentals: never trusting client-side checks, never logging or storing sensitive data in plain text, validating all input including at the native code boundary, and locking down exported components by default.

This exercise forms part of ongoing, hands-on practice in Android application security testing and mobile penetration testing methodology.